

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE CODE: CSA0409

COURSE NAME: Operating Systems

COURSE OUTCOME COVERED

CO4: *Optimize various file allocation strategies and disk scheduling algorithms to identify optimal methods for enhancing system performance.*

Assessment Tool Weightage: 30%

QUESTIONS: 5

Total Marks:25

Annexure A

COMPARATIVE ANALYSIS

Code of Conduct:

I k.jaya venkata sai / 192525182 *certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

k.jaya venkata sai

Comparative Analysis

Name : K. Jaya Venkata Sai

Reg No : 192525182

Subject : OS

Code : CSA0409

C04 - AT3

Date : 13-8-26.

Contiguous Vs Linked File Allocation.

Feature	Contiguous Allocation	Linked Allocation
Access speed	Very fast	Slow
Sequential access	Excellent	Good
Direct access	Excellent	Poor
Memory utilization	May cause fragmentation	Better utilization
Fragmentation	External fragmentation	No External fragmentation
Overhead.	Low	Pointer overhead.

- * Contiguous allocation stores file blocks continuously, so it provides faster access and less disk-head movement.
- * Linked allocation stores block anywhere on the disk and connects them using pointers. It avoids external fragmentation but requires pointer traversal.

Best for Sequential file access :- Contiguous Allocation because consecutive blocks can be accessed quickly.

2.

Linked Vs Indexed file Allocation.

Feature	Linked Allocation	Indexed Allocation
Random Access	Poor	Good
Storage overhead	Pointer in each block	Index block required
Large files	Suitable	Highly suitable
Reliability	Pointer damage	Index damage
Flexibility	High	High
Access Speed.	Low	Higher for random access.

* Linked allocation uses pointers to connect file blocks. It has lower management complexity but random access is slow.

* Indexed allocation maintains an index block containing addresses of all file blocks. It provides efficient direct / random access.

More efficient for large files : Indexed allocation especially when files require frequent random access. It can efficiently locate data blocks with traversing a chain of pointers.

FCFS Vs SSTF Disk Scheduling

Feature	FCFS	SSTF
Selection method	Arrival order	Nearest request
Seek time	Usually high	Usually lower
Fairness	No Very high	Lower
Starvation	No	Possible
Performance	Lower	Better
Heavy disk load.	Less efficient	More efficient

- FCFS (First come first serve) services requests in their arrival order. It is simple and fair but may cause large head movements.

- SSTF (Shortest Seek Time-first) selects the requests closest to the current head position. It generally reduces seek time and improves performance.

Conclusion : SSTF is more efficient for performance, while FCFS is better when fairness and simplicity are the main requirements.

Scan Vs C-Scan Disk Scheduling.

Feature	Scan	C-Scan
Head movement	Both directions	one serving dir.
Response time	Good	more uniform
Fairness	Good	Very Good
Head Movement	can be higher	more Predictable
Waiting time	Variable	more uniform
Real-time suitability	Good	Better

* Scan moves the disk head in one direction, services requests, and then reverses direction.

* C-Scan services requests in one direction only. After reaching the end, the head returns to the beginning and starts servicing again.

Conclusion :- for Systems requiring Predictable and Consistent Service, C-Scan is Preferred over Scan.

5

Look Vs C-Look Disk Scheduling.

Feature	Look	C-Look
Direction	Both directions	one direct
Endpoint Movement	Does not go to physical end	Jumps to ^{request.} next pending
Total head Movement	lower than SCAN	usually lower
Waiting Time	Good	More uniform
Efficiency	High	Very High
Implementation	Relatively Simple	Slightly More Complex

→ Look is similar to SCAN, but the disk head reverses direction when there are no more requests in the current direction. It does not travel to the physical end of the disk unnecessarily.

→ C-Look is similar to C-SCAN. It services requests in one direction and after reaching the last request, jumps back to the first pending request.

Conditions :-

- Look : Better when balanced access and simple implementation are required.
- C-Look : Better when uniform response time and predictable services are important.

Conclusion : Both are more efficient than SCAN / C-SCAN because they avoid unnecessary travel to the disk boundaries. For predictable performance, C-Look is Preferred.